Implementation of Single-Agent Search Algorithms for Asteroid Escape

José del Solar Zavala
Doctorado en Sistemas de Ingeniería
Universidad de Talca
Talca, Chile
jsolar18@alumnos.utalca.cl

I. Asteroid Escape

Sliding puzzles are a type of game where the objective is to move items in a constrained grid towards a clear goal [1]. *Asteroid Escape* is a puzzle game that fits this criteria. It is played on a 3x3 board and the objective is as it was described in general for sliding puzzle games.



Fig. 1. Every piece in Asteroid Escape base game

The rules for Asteroid Escape are stipulated on the Official Challenge Booklet published by SMART Games alongside the game:

1) Select a challenge. Place the 8 puzzle pieces on the game board as indicated. Pay attention to the exact orientation of each puzzle piece!
2) Slide the puzzle pieces, one by one:
   - All puzzle pieces must remain horizontal on the game board! Tilting or lifting up pieces is not allowed.
   - Sometimes asteroids will overlap empty parts of other puzzle pieces or the border of the game board. But asteroids can never overlap other asteroids!
   - The spaceship can also overlap empty parts of puzzle pieces. But the spaceship can never overlap asteroids.
   - Only the spaceship can slide off the game board.
3) You have found the solution when you can slide the spaceship through the opening of the game board. The shortest solution is shown at the end of the game booklet.



Fig. 2. Asteroid Escape example board.

These are the rules used to construct a simulated environment and a solver based on different single-agent search algorithms. This is an interesting case study because the second rule has conditions that restrict movement and makes the representation less trivial than other more well-known sliding puzzle games such as *Rush Hour* or *15-puzzle*.

The challenge booklet also includes a list of 60 total different puzzles of varying difficulties: junior, starter, expert, master and wizard, all 60 are provided with the solution so it serves as a good basis to test the solver and the algorithm implementations.

II. Algorithms Implemented

In this section, the algorithms used to solve instances of *Asteroid Escape* are briefly described. The different heuristics implemented are also described as a way to give context to some of the algorithms that require the use of heuristics.

A. Breadth-First Search

Breadth-First Search (BFS) is one of the simpler search methods that can be applied in puzzle game solving. As it name implies, it explores all the nodes from a game tree, starting from an initial state and exhaustively exploring the nodes at the current depth level of the tree. This can be quite costly in memory compared to some other algorithms as the entire set of reachable states from the initial has to be

stored, which is not necessarily a bad thing depending on the computing resources available.

In the case for the simulation of *Asteroid Escape*, there is only one initial state and each legal move is simulated, then each of those resulting states from applying the legal moves will also have their legal moves generated and explored, iteratively repeating the process until a solution is reached, as the container used to store the nodes, both a set and a queue are used, because looking for an existing node in a set is a constant operation, which is not the case for queues, also popping the next node to be evaluated in a queue is a constant operation while the set is unordered.

B. A* search algorithm

A* is one of the most popular informed pathfinding algorithms, meaning that it's used to find the shortest path between two points in a grid, by definition this already sounds like an improvement over BFS, where all the nodes are explored without any approximation until a solution is found. A* works by doing what is called a best-first search, meaning that the nodes will be expanded based on a heuristic.

To do this, A* works by keeping two different lists called OPEN and CLOSED. On each iteration, the node in OPEN with the lowest total cost (cost of moving to that node + heuristic value) is evaluated, in the same way as the BFS implementation, the legal moves are generated and simulated. If a resulting node has the following conditions it is added to the OPEN list for further exploration:

- If the resulting node is not already in the OPEN list.
- If the resulting node was already in either the CLOSED or OPEN lists, but the previous total cost is higher than the current evaluated total cost (also called a reopening of nodes).

Otherwise, the node will be added to the CLOSED list. This process is repeated until a solution is found. By the nature of the algorithm described, it will explore on most occasions less nodes than the BFS implementation, being dependant on a good heuristic. However, if the branching factor of our game is high, then, like with BFS, the amount of nodes that have to be kept in memory grows a lot, which can be a problem for limited computational power.

For the implementation of *Asteroid Escape* the same strategy of using sets to have a constant operation that checks the nodes was used, and a priority queue that automatically orders the nodes in it by their total cost.

C. Iterative-deepening A*

Iterative-deepening A*, as it name implies is an informed depth-first search (DFS) that borrows the strategy of using a heuristic to guide the search from A*. The algorithm works by giving the search an initial depth limit based on a heuristic estimate, then performs a depth-first search until the limit is reached, while recording the minimum cost found at that limit, if during the search the solution is encountered, then the algorithm terminates. If the algorithm reaches the depth limit but doesn't find the solution, then the depth limit is increased and the process is repeated.

By the nature of the algorithm described, it is understandable how IDA* only needs to keep the current path stored in memory instead of the whole game tree, making it efficient memory-wise compared to A* and BFS. However, this comes at the cost of not having a direct way to check and control for duplicity meaning that repeated nodes are expanded multiple times across the execution of IDA*.

Having this memory efficiency makes IDA* a more desirable algorithm to use in large search spaces where the amount of nodes that would have to be kept in memory in A* and BFS become a problem to the memory available, however it is not necessarily faster in all cases to the other two algorithms because it is more prone to expand repeated nodes.

D. Heuristics

Since A* and IDA* are dependant on an heuristic which affects the way and amount of nodes that are explored, it is a part of the job to design an admissible heuristic (meaning that the value of the heuristic of a node can't be higher than the actual cost of a node). Measuring the cost of the node is simple for Asteroid Escape as looking at the moves for cost is the only measure needed.

Four different heuristics are proposed, a lot of them build from the previous one.

- **ZERO:** This means that the heuristic value is always zero for any node.
- **TWOONE:** If the spaceship is not on the goal column, then there is at least one move needed to reach the correct column. Also, if the spaceship is not on the goal row, then there is at least one move needed to reach the goal row.
- **OBSTACLES0:** The TWOONE heuristic plus if the piece in front of the goal is not the spaceship nor is it an empty piece, then there's at least one move required to unblock the exit.
- **OBSTACLES1:** The TWOONE heuristic plus same logic as OBSTACLES0 but considering every piece on the same column of the goal that is on a row below the spaceship. Basically, for each piece that is not empty between the spaceship and the goal, there's at least one move needed to unblock it.

III. Results

In this section, a series of tables with interesting stats to discuss will be presented in order to be discussed. The simulation was programmed using standard C++ libraries. The experiments ran on a WSL Ubuntu 22.04.2 LTS distribution installed on a Windows 11 22H2 machine with 16GB of RAM and a AMD Ryzen 7 6800H processor (3.20 GHz). The code was run with O3, msse and msse2 compilation flags.

The experiments consisted on running every algorithm with different heuristics if applicable on the boards 55 to 60, which are classified as difficulty *wizard*. Some interesting data shown in the tables include: time, nodes expanded (called total iterations) and nodes per second. There are some algorithm

specific stats as well. For example, the IDA* tables have a column called "total iterations at max limit" which refers to the total nodes expanded at the bound of the solution length. For example, if the solution length is 10, then that column represents the nodes expanded when the depth limit is 10.

Board (Solution Length)	Time (s)	Total Iterations	Open list size	Nodes/Second
Wizard 55 (52)	0.088	34,822	132	497,457
Wizard 56 (60)	0.125	56,275	681	618,407
Wizard 57 (93)	0.031	18,088	1,054	602,933
Wizard 58 (77)	0.027	17,821	1,000	594,033
Wizard 59 (94)	0.017	9,824	426	654,933
Wizard 60 (109)	0.052	25,671	623	513,420

TABLE I
RESULTS FROM BFS.

A. Results from A*

Board (Solution Length)	Time (s)	Total Iterations	Closed List Size	Nodes/Second
Wizard 55 (52)	0.069	34,735	16,233	503,406
Wizard 56 (60)	0.108	56,965	25,748	542,524
Wizard 57 (93)	0.036	18,452	8,112	527,200
Wizard 58 (77)	0.041	18,265	7,817	493,649
Wizard 59 (94)	0.018	9,821	4,490	545,611
Wizard 60 (109)	0.069	26,178	11,595	402,738

TABLE II
RESULTS FROM A* (HEURISTIC=0)

Board (Solution Length)	Time (s)	Total Iterations	Closed List Size	Nodes/Second
Wizard 55 (52)	0.07	34,332	16,019	497,565
Wizard 56 (60)	0.1	54,708	24,703	516,113
Wizard 57 (93)	0.027	13,892	6,104	534,308
Wizard 58 (77)	0.025	13,040	5,559	465,714
Wizard 59 (94)	0.014	7,956	3,642	530,400
Wizard 60 (109)	0.058	23,945	10,552	412,845

TABLE III
RESULTS FROM A* (HEURISTIC=TWOONE)

Board (Solution Length)	Time (s)	Total Iterations	Closed List Size	Nodes/Second
Wizard 55 (52)	0.067	34,082	15,899	501,206
Wizard 56 (60)	0.1	53,226	24,006	526,990
Wizard 57 (93)	0.024	12,425	5,421	540,217
Wizard 58 (77)	0.023	11,772	5,027	535,091
Wizard 59 (94)	0.015	7,800	3,574	520,000
Wizard 60 (109)	0.055	22,039	9,662	415,830

TABLE IV
RESULTS FROM A* (HEURISTIC=OBSTACLES0)

Board (Solution Length)	Time (s)	Total Iterations	Closed List Size	Nodes/Second
Wizard 55 (52)	0.067	33,882	15,790	505,701
Wizard 56 (60)	0.105	53,255	23,998	537,929
Wizard 57 (93)	0.022	12,025	5,260	546,591
Wizard 58 (77)	0.021	10,618	4,509	530,900
Wizard 59 (94)	0.014	7,536	3,439	538,286
Wizard 60 (109)	0.048	20,698	9,016	413,960

TABLE V
RESULTS FROM A* (HEURISTIC=OBSTACLES1)

B. Results from IDA*

Board (Solution Length)	Time (s)	Total Iterations	Total Iterations at max limit	Nodes/Second
Wizard 55 (52)	284.978	110,362,666	10,498,278	423,291
Wizard 56 (60)	307.491	114,557,425	703,697	389,577
Wizard 57 (93)	4.074	1,651,905	14,964	405,475
Wizard 58 (77)	0.717	271,488	25,646	372,412
Wizard 59 (94)	0.824	311,530	67,436	378,070
Wizard 60 (109)	6.484	2,337,128	174,191	367,936

TABLE VI
RESULTS FROM IDA* (HEURISTIC=ZERO)

Board (Solution Length)	Time (s)	Total Iterations	Total Iterations at max limit	Nodes/Second
Wizard 55 (52)	175.687	66,473,338	6,310,448	378,362
Wizard 56 (60)	166.33	64,410,607	408,486	387,246
Wizard 57 (93)	3.593	1,445,116	10,783	402,203
Wizard 58 (77)	0.481	191,216	14,747	387,077
Wizard 59 (94)	0.527	199,897	42,775	379311
Wizard 60 (109)	3.92	1,450,751	104,308	370,090

TABLE VII
RESULTS FROM IDA* (HEURISTIC=TWOONE)

Board (Solution Length)	Time (s)	Total Iterations	Total Iterations at max limit	Nodes/Second
Wizard 55 (52)	136.94	51,837,767	4,962,650	383,175
Wizard 56 (60)	126.84	47,943,697	298,317	377,986
Wizard 57 (93)	3.556	1,348,664	9,028	393,771
Wizard 58 (77)	0.416	162,796	10,583	376,843
Wizard 59 (94)	0.437	168,869	36,900	357,017
Wizard 60 (109)	3.127	1,140,456	75,997	360,790

TABLE VIII
RESULTS FROM IDA* (HEURISTIC=OBSTACLES0)

Board (Solution Length)	Time (s)	Total Iterations	Total Iterations at max limit	Nodes/Second
Wizard 55 (52)	120.437	45,300,943	4,875,770	382,106
Wizard 56 (60)	98.639	36,473,376	239,534	369,766
Wizard 57 (93)	3.14	1,272,276	8,038	403,129
Wizard 58 (77)	0.379	141,949	8,131	368,699
Wizard 59 (94)	0.517	193,114	41,817	369,950
Wizard 60 (109)	2.533	920,327	60,773	363,335

TABLE IX
RESULTS FROM IDA* (HEURISTIC=OBSTACLES1)

IV. DISCUSSION

First, it is important to reminder that each heuristic builds from the previous one (OBSTACLES0 from TWOONE, OBSTACLES1 from OBSTACLES0), and it's clear that they do work in reducing the amount of nodes that every algorithm expands. This is clearly evident in tables that describe the results from the IDA* where the time gained comparing the use of no heuristic and the different heuristics.

It is also noticeable that IDA* is the one that took the most time in reality to solve the boards, especially on the boards 55 and 56, followed by BFS and then A*. The slowness of IDA* in this implementation could be attributed to the large amount of duplicate nodes in the search space. If the nature of Asteroid Escape is analyzed, it is possible to note that there is not a large branching factor (around 4 for the maximum available moves), therefore it is very likely that IDA* gets trapped in duplicate loops, the search space of the boards analyzed is also not big (these can be checked on Table 1 or Table 2). For this reason, at least for the analyzed nodes, IDA* is not that useful compared to A*. However, it is known that for problems that would make the open list from A* grow at a rapid rate and by a large quantity then IDA* would be useful as it does not use memory. One characteristic is that even if in some cases

is slower, you can guarantee that sooner or later IDA* will eventually find the solution, but that's not always the case for A*.

Another factor to take into account is that the BFS implementation has the best Nodes/Seconds factor for every board analyzed. This can be because the heuristic does not need to be calculated, the nodes do not need to be arranged in the open list, like in A*, also there are no recursive calls and not too many duplicate nodes like in IDA*. However, it is still slower overall than A*, because the later expands fewer nodes thanks to the heuristics.

V. Conclusion

This short report detailed the results obtained from a C++ simulation of the game Asteroid Escape from SMART Games. Three different single-agent search algorithms were implemented and studied: BFS, A* and IDA*. The boards studied come from the challenge booklet provided with the physical game and online on the official game page. From the results obtained, A* using a heuristic was the fastest in run time because the speed of a search algorithm is related to the amount of nodes it expands, despite A* needing more operations per node as BFS. It is worth noting that on more complex environments, especially on larger search spaces IDA* is proposed to be more useful than A* as the first does not require to kept paths in memory like the later.

In future work, adding more different pieces or increasing the size of the boards could be a good experiment to see at which point IDA* starts to become more useful than A*.

References

[1] B. De Kegel and M. Haahr, "Procedural puzzle generation: A survey," *IEEE Transactions on Games*, vol. 12, no. 1, pp. 21–40, 2019.